

Physics-Informed Neural Operators for Option Pricing

Zefang Li^a, Zhan Liu^a

^aDepartment of Civil and Systems Engineering, Johns Hopkins University, 3400 N. Charles Street, Baltimore, 21218, Maryland, USA

ARTICLE INFO

Keywords:
Option pricing
Neural operators

ABSTRACT

Traditional partial differential equation (PDE) methods for option pricing are computationally expensive. To accelerate this, we propose a physics-informed neural operator framework to approximate the Black-Scholes PDE. We evaluate Fourier Neural Operators (FNO) and DeepONet against Convolutional Neural Operators (CNO) and a Multi-Layer Perceptron (MLP) using a dual-network architecture that maps implied volatility surface, moneyness, and time-to-maturity to normalized prices. Ablation studies unexpectedly reveal that removing boundary condition constraints significantly improves model optimization. Our fine-tuned FNO model achieves state-of-the-art performance on real-world data, yielding a test mean squared error (MSE) of 0.000373 and an R^2 of 0.9996. Demonstrating robust accuracy across various volatility regimes and pricing over 24,000 options per second, this FNO framework proves highly viable for real-time quantitative finance and risk management.

1. Introduction and Related Work

Options are an important derivative in financial markets that gives the buyer the right, but not the obligation, to buy or sell a security at a specific strike price K in dollars after a certain time T in years. Traditional option pricing often relies on numerical methods, such as finite difference schemes, to solve partial differential equations (PDEs). However, these methods can become computationally expensive when modeling dynamic, high-dimensional market states. Therefore, an efficient surrogate model is urgently needed to replace expensive numerical solvers while maintaining acceptable accuracy.

Recent advances in neural operator learning have introduced powerful alternatives to classical numerical methods. DeepONet [1] is built on the universal approximation theorem of operators and uses a branch-trunk architecture to learn mappings between infinite-dimensional function spaces. Fourier Neural Operator (FNO) [2] offers a resolution-invariant approach by operating directly in the Fourier domain and achieves superior performance on parametric PDEs. Convolutional Neural Operator (CNO) [3] preserves spatial structure through convolutional layers while maintaining continuous-discrete equivalence and excellent out-of-distribution generalization. With advances in neural network structures, multiple attempts have been started to utilize them for option pricing [4, 5, 6], while the capabilities of neural operators are not yet fully utilized.

In this work, we develop neural operator networks that learn the underlying mathematical mapping between the state of the market and the fair price for European call options. Once trained, the model can efficiently evaluate option prices and their corresponding Greeks at any underlying asset price or time-to-maturity, completely independent of the original input grid resolution. Ultimately, applying neural operators to quantitative finance significantly accelerates real-time derivative pricing and risk management without sacrificing the rigor of established mathematical models.

European call option pricing follows the Black-Scholes (BS) PDE, as shown in Equation 1, where $V(S, t) \in \mathbb{R}$ is the value of the option, $r \in \mathbb{R}$ is the risk-free interest rate, $q \in \mathbb{R}$ is the dividend rate, $S \in \mathbb{R}$ is the stock price, and $\sigma \in \mathbb{R}$ is the volatility.

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q)S \frac{\partial V}{\partial S} - rV = 0 \quad (1)$$

For a European call option with strike K and maturity time T , the relevant boundary conditions are:

- Terminal condition: $V(S, T) = \max(S - K, 0)$
- Lower boundary: $V(0, t) = 0$

 zli299@jhu.edu (Z. Li); zliu274@jh.edu (Z. Liu)

ORCID(s):

 zli299@jhu.edu (Z. Li); zliu274@jh.edu (Z. Liu)

ORCID(s):

- Upper boundary: $\lim_{S \rightarrow \infty} V(S, t) = S \exp(-q(T - t)) - K \exp(-r(T - t))$

The BS PDE has an analytical solution if σ is constant, but this ideal case is rarely true in real markets. In practice, σ is unknown and highly dependent on the current state of the market as well as S and the time to execution $\tau = T - t$. Denoting the market state $u \in \mathbb{R}^m$ with m dimensions, we seek to find the solutions for $\sigma(S, \tau; u)$ and $V(S, \tau, \sigma)$ simultaneously. We require σ to be an input to the price model V , because the derivative Vega $v = \frac{\partial V}{\partial \sigma}$ is practically useful.

To stabilize neural network training, we nondimensionalize Equation 1 to limit the ranges of the inputs and outputs. We learn the normalized value $v = \frac{V}{K}$ as a function of moneyness $M = \frac{S}{K}$ and the annualized time to execution $\tau = T - t$. Because τ is now expressed in years, the time derivative transforms via the chain rule as $\frac{\partial V}{\partial t} = -K \frac{\partial v}{\partial \tau}$. Substituting this into the original PDE and dividing by the strike price K , Equation 1 becomes Equation 2:

$$-\frac{\partial v}{\partial \tau} + \frac{1}{2}\sigma^2 M^2 \frac{\partial^2 v}{\partial M^2} + (r - q)M \frac{\partial v}{\partial M} - rv = 0 \quad (2)$$

The corresponding boundary conditions update to the following:

- Terminal condition: $v(M, \tau = 0) = \max(M - 1, 0)$
- Lower boundary: $v(0, \tau) = 0$
- Upper boundary: $\lim_{M \rightarrow \infty} v(M, \tau) = M \exp(-q\tau) - \exp(-r\tau)$

These serve as the physical foundation for the model and are enforced as soft constraints during training.

2. Methodology

2.1. Data Acquisition & Pre-processing

The data used for training and testing are downloaded from IvyDB US by OptionMetrics, provided by Wharton Research Data Services (WRDS). The dataset, ranging from August 29, 2024, to August 29, 2025, contains daily data for SPX call option prices (V), SPX prices (S), SPX dividend yields (q), interest rates (r), and implied volatility surfaces ($[\sigma_{\text{imp}}]_{11 \times 34}$).

To pre-process the data, we utilize the Polars library to efficiently parse and compile the raw market data into a single dataset:

- Filtering Criteria: The dataset is restricted exclusively to Call options. We filter out illiquid options by requiring an open interest greater than 100 and a best bid greater than 0.
- Feature Calculation: We compute dimensionless financial metrics, including the mid price (mean of best bid and offer), normalized price ($v = \frac{V}{K}$), moneyness ($M = \frac{S}{K}$), and time to Maturity ($\tau = T - t$).
- Data Alignment: Datasets are merged by trade dates. Interest rates are aligned using a nearest-neighbor approach based on days to expiry, while dividends are joined on both trade date and expiration date.

The second stage transforms the structured tabular data into vectorized inputs suitable for operator training. We partition the data into specific NumPy arrays:

- Branch Input (U): A flattened vector representation of the implied volatility surface $[\sigma_{\text{imp}}]_{11 \times 34}$, resulting in a 374-dimensional input.
- Trunk Input (Y): A 4-dimensional continuous feature array consisting of $[M, \tau, r, q]$.
- Target (v): The normalized option price.

The total dataset is split chronologically to guarantee isolation of the test data. The first 80% of dates (640,582 samples) are used for training, the following 10% (80,073 samples) for validation, and the final 10% (80,073 samples) for testing.

2.2. Physics-Informed Loss Function Formulation

We adopt physics-informed machine learning, requiring the model to fit real market data while satisfying the BS PDE (Equation 2) and its boundary conditions. The composite loss function includes:

- Data Loss: Mean squared error against observed market prices.

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} (\hat{v}_i - \hat{v}_{\text{true},i})^2 \quad (3)$$

- PDE Loss: Penalizing the residual $\mathcal{R}$ of Equation 2, where $\mathcal{R} = -\frac{\partial \hat{v}}{\partial \tau} + \frac{1}{2}\hat{\sigma}^2 M^2 \frac{\partial^2 \hat{v}}{\partial M^2} + (r - q)M \frac{\partial \hat{v}}{\partial M} - r\hat{v}$.

$$\mathcal{L}_{PDE} = \frac{1}{N_{colloc}} \sum_{i=1}^{N_{colloc}} \mathcal{R}_i^2 \quad (4)$$

- Terminal Condition Loss: Enforcing the payoff at maturity.

$$\mathcal{L}_{term} = \frac{1}{N_{term}} \sum (\hat{v}(M, 0) - \max(M - 1, 0))^2 \quad (5)$$

- Lower Boundary Loss: Call value approaches 0 as $M \rightarrow 0$.

$$\mathcal{L}_{lower} = \frac{1}{N_{lower}} \sum (\hat{v}(0, \tau))^2 \quad (6)$$

- Upper Boundary Loss: Approximated at $M_{max} = 5.32$, the maximum dataset value.

$$\mathcal{L}_{upper} = \frac{1}{N_{bndry}} \sum (\hat{v}(M_{max}, \tau) - (M_{max}e^{-q\tau} - e^{-r\tau}))^2 \quad (7)$$

- No-Arbitrage Loss: Enforcing monotonicity ($\partial_M \hat{v} \geq 0$) and convexity ($\partial_{MM} \hat{v} \geq 0$) using ReLU penalties.

$$\mathcal{L}_{arb} = \frac{1}{N_{colloc}} \sum \left(\text{ReLU} \left(-\frac{\partial \hat{v}}{\partial M} \right) + \text{ReLU} \left(-\frac{\partial^2 \hat{v}}{\partial M^2} \right) \right) \quad (8)$$

The total loss is the weighted sum of these components:

$$\mathcal{L}_{total} = \lambda_{data} \mathcal{L}_{data} + \lambda_{PDE} \mathcal{L}_{PDE} + \lambda_{term} \mathcal{L}_{term} + \lambda_{lower} \mathcal{L}_{lower} + \lambda_{upper} \mathcal{L}_{upper} + \lambda_{arb} \mathcal{L}_{arb} \quad (9)$$

where we set $\lambda_{data} = 1.0$, $\lambda_{PDE} = \lambda_{term} = \lambda_{lower} = \lambda_{upper} = \lambda_{arb} = 0.5$ as a starting point to softly prioritize fitting the real market data.

2.3. Model Formulation & Optimization

The solver consists of two interconnected networks (Figure 1). Network 1 maps the market state (σ_{imp}), M , and τ to the true volatility σ . Network 2 takes σ , r , q , M , and τ to calculate the normalized value v . This automatic differentiation framework guarantees the exact computation of the Greeks: Delta ($\Delta = \frac{\partial V}{\partial S}$), Gamma ($\Gamma = \frac{\partial^2 V}{\partial S^2}$), Theta ($\Theta = \frac{\partial V}{\partial t}$), Vega ($\nu = \frac{\partial v}{\partial \sigma}$), and Rho ($\rho = \frac{\partial V}{\partial r}$).

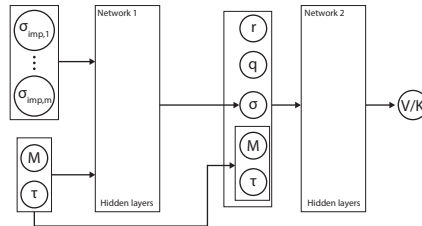


Figure 1: Network structure diagram showcasing the coupled volatility and pricing models.

Network 2 is a standard fully connected neural network solving the known PDE structure. Network 1 is modeled as a deep neural operator to handle the complex volatility mapping. Specifically, we define a solution operator $\mathcal{G}$ mapping the local volatility function to the option price surface. The FNO or DeepONet approximates this using a branch and trunk architecture:

$$\sigma(S, t) \approx \mathcal{G}(\sigma_{imp})(S, t, r, q) = \sum_{k=1}^p b_k(\sigma_{imp}) \cdot t_k(S, t, r, q) \quad (10)$$

where b_k and t_k are the outputs of the branch and trunk networks, respectively, and $p = 64$ is the latent dimension. We also build a Fourier Neural Operator (FNO), where the 374-dimensional U vector is converted back to its original shape $[\sigma_{\text{imp}}]_{11 \times 34}$ on a grid of 11 Days and 34 Deltas as in the dataset before pre-processing, and then passed through 4 FNO layers before the dot product of the branch net and the trunk net is taken. For comparative benchmarking, Network 1 is also trained as a vanilla Multi-Layer Perceptron (MLP), and a Convolutional Neural Operator (CNO) where convolution layers are added to the branch net.

Optimization is performed using Adam in two stages: an initial exploratory phase with a learning rate of 10^{-3} under a schedule with a factor of 0.5 and a patience of 3, followed by fine-tuning at a constant learning rate of 10^{-5} with a patience of 3.

During training, the total loss $\mathcal{L}_{\text{total}}$ on the training set is used for back-propagation and only the data loss $\mathcal{L}_{\text{data}}$ is used on the validation set to decide whether to save the best model so far and if an early stop is needed. The final test on the model is performed on the test set with only data loss $\mathcal{L}_{\text{data}}$.

3. Results and Analyses

Preliminary findings indicate that the physics-informed architecture successfully balances empirical data fitting with theoretical PDE constraints. Figure 2 illustrates the training, validation, and testing loss trajectories across the four tested architectures. With the same set of training parameters and the same trunk net of Network 1 and the same Network 2, adopting DeepONet and FNO for the branch net of Network 1 yields the lowest test errors of 0.001578 and 0.000686, respectively, while the CNO and vanilla MLP give higher test errors of 0.001188 and 0.002475. This preliminary results indicate that FNO and DeepONet outperforms the CNO and MLP benchmarks, and thus further investigations are focused on FNO and DeepONet.

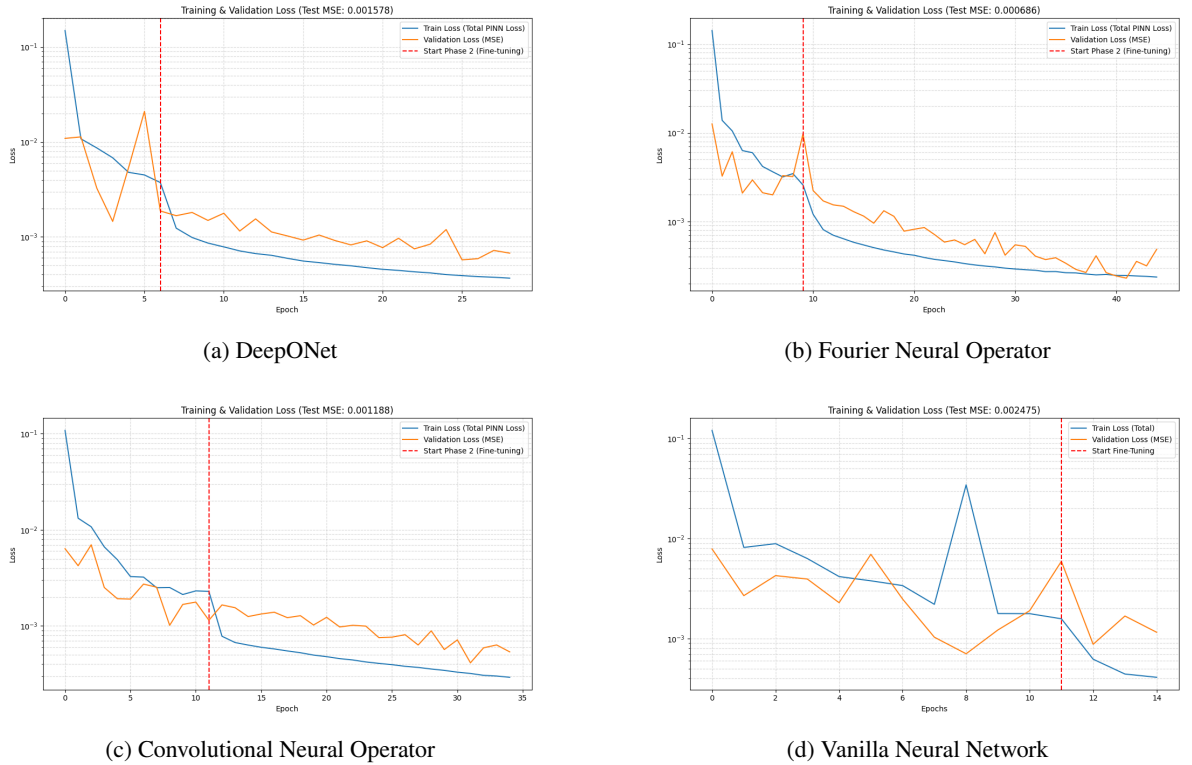


Figure 2: Training, validation, and testing loss trajectories across the four tested architectures.

3.1. Loss Weight Sensitivity Analysis

To investigate the contribution of each physics-informed loss component to model performance, we conduct an ablation study for both DeepONet and FNO by systematically removing individual loss terms from the full loss function. Figure 3 summarizes the five configurations tested: the full baseline with all loss components enabled

($\lambda_{data} = 1.0$, $\lambda_{PDE} = 0.5$, $\lambda_{term} = 0.5$, $\lambda_{lower} = 0.5$, $\lambda_{upper} = 0.5$, $\lambda_{arb} = 0.5$), and four ablated variants that respectively disable the PDE residual loss (-PDE), boundary condition losses (-BC), arbitrage penalty (-Arb), or all physics-informed terms (DataOnly). As shown in Figure 4, our results reveal a counterintuitive finding: removing the boundary condition losses (-BC) consistently yields the best performance across both architectures, achieving test MSE of 0.000535 for DeepONet (67.2% improvement over baseline) and 0.000373 for FNO (80.7% improvement over baseline). This surprising result suggests that the boundary condition constraints may introduce conflicting gradients during optimization, potentially because the neural network already captures boundary behavior implicitly through the data-driven loss term. Based on these findings, we adopt the 'No Boundaries' configuration as the optimal loss setting for both DeepONet and FNO architectures and proceed to conduct hyperparameter analysis and tuning in the following section.

Configuration	λ_{data}	λ_{PDE}	λ_{term}	λ_{lower}	λ_{upper}	λ_{arb}	Label
Baseline	1.0	0.5	0.5	0.5	0.5	0.5	Full
No PDE	1.0	0.0	0.5	0.5	0.5	0.5	-PDE
No Boundaries	1.0	0.5	0.0	0.0	0.0	0.5	-BC
No Arbitrage	1.0	0.5	0.5	0.5	0.5	0.0	-Arb
Data Only	1.0	0.0	0.0	0.0	0.0	0.0	DataOnly

Figure 3: Loss weight configurations for sensitivity analysis.



Figure 4: Comparison between DeepONet and FNO across different loss configurations.

3.2. Hyperparameter Analysis and Tuning

We investigate the impact of network depth on model performance by varying the number of hidden layers from 2 to 6 for both architectures. As illustrated in Figure 5, DeepONet achieves optimal performance with 2 or 4 hidden layers (MSE = 0.000535), while increasing to 6 layers leads to significant performance degradation (MSE = 0.002018), which suggests potential overfitting or optimization difficulties with deeper networks. In contrast, FNO achieves the best performance with 4 hidden layers (MSE = 0.000373), and deeper (6 layers, MSE = 0.000404) configuration yields suboptimal results. These findings indicate that DeepONet favors shallower architectures, whereas FNO requires moderate depth to effectively capture the underlying operator mapping.

Building upon the 4-layer architecture identified in the previous analysis, we further examine the effect of hidden layer width by varying the number of neurons per layer from 16 to 64. As shown in Figure 6, both architectures exhibit a consistent trend: increasing the number of neurons leads to monotonically decreasing test MSE. For DeepONet, expanding from 16 to 64 neurons reduces the MSE from 0.003298 to 0.000535 (83.8% improvement), while FNO demonstrates an even more significant improvement from 0.003548 to 0.000373 (89.5% improvement). These results suggest that wider networks provide greater representational capacity and enable both models to better approximate the complex nonlinear mapping inherent in option pricing.

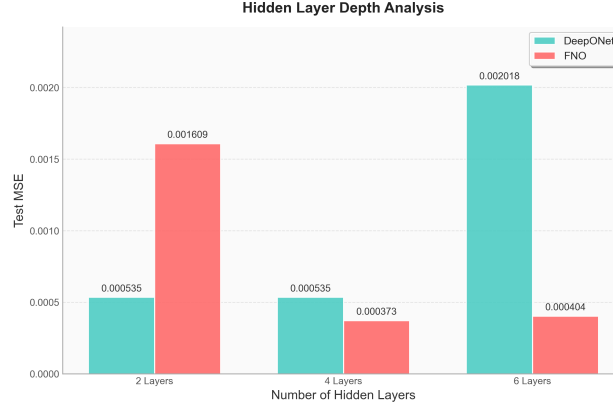


Figure 5: Impact of hidden layer depth on model performance for DeepONet and FNO.

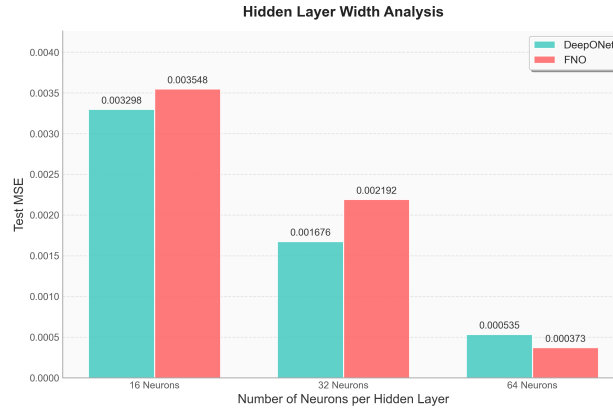


Figure 6: Impact of hidden layer width on model performance for DeepONet and FNO.

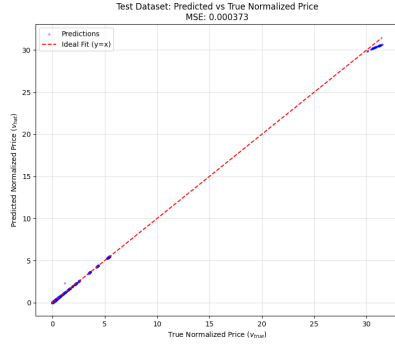
3.3. Final Model Performance Analysis

The best model with lowest final test loss is the FNO model where the trunk net has 4 layers with 64 neurons each, trained with $\lambda_{data} = 1.0$, $\lambda_{PDE} = \lambda_{arb} = 0.5$, and $\lambda_{term} = \lambda_{lower} = \lambda_{upper} = 0$. The final test loss is 0.000373 for this model and its performance is further analyzed.

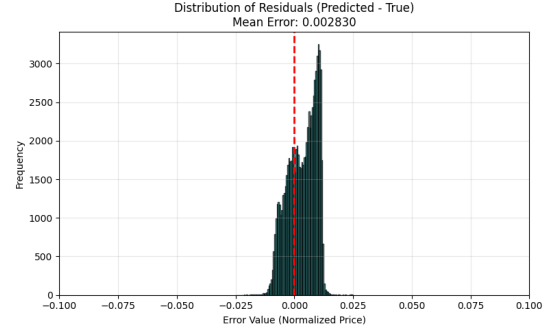
3.3.1. Global error and distribution analysis

The primary objective of this experiment is to establish the baseline accuracy of the model across the entire test dataset. Standard machine learning metrics are computed between the network's predicted normalized option price (v_{hat}) and the true normalized price (v_{true}).

The mean squared error (MSE) and root mean squared error (RMSE) are evaluated to capture the variance of the residuals and penalize large deviations. We obtain for the test set: $MSE = 0.000373$, $RMSE = 0.019309$. The R^2 score is also calculated to quantify the proportion of the variance in the dependent variable that is predictable from the independent variables. We have $R^2 = 0.999631$, which is very close to 1, indicating a good estimation of the normalized price. The predicted results and ground truth for the entire test set shows consistent prediction and more error for those options with higher values (Figure 7a). Additionally, the distribution of the residuals ($\hat{v} - v_{true}$) is evaluated and its distribution is shown in Figure 7b. It exhibits a tight distribution of residuals with a mean error of 0.002830, indicating that the model tends to slightly overestimate the value of call options. The distribution is distorted with multiple peaks.



(a) Predicted value from the final model (v_{hat}) and ground truth v_{true} for the entire test set.



(b) Residual ($\hat{v} - v_{true}$) distribution histogram.

Figure 7: Prediction performance of the final FNO model.

3.3.2. Error stratification heatmap (moneyness vs. maturity)

Financial models rarely fail uniformly. They tend to exhibit varying performance across different market regimes. This experiment stratifies the pricing error (MAE) across a two-dimensional grid defined by moneyness ($M = S/K$) and time to maturity (τ). The test dataset is binned into the following categories:

- Moneyness (M): Deep out-of-the-money (OTM) ($M < 0.95$), OTM ($0.95 \leq M < 0.98$), At-the-money (ATM) ($0.98 \leq M < 1.02$), In-the-money (ITM) ($1.02 \leq M < 1.05$), and Deep ITM ($M \geq 1.05$).
- Maturity (τ): Short-term ($\tau < 0.1$ years), Medium-term ($0.1 \leq \tau < 0.5$ years), and Long-term ($\tau \geq 0.5$ years).

The resulting heatmap indicates that the model has the highest error for the most for medium-term out-of-the money and short-term at-the-money options.

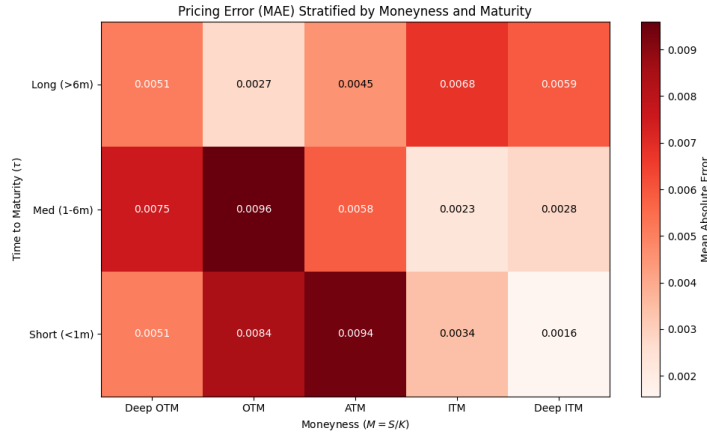


Figure 8: Error heatmap for different moneyness and time to maturity combinations.

3.3.3. The volatility's effect on error

The FNO demonstrates high invariance to market volatility regimes. By feeding the daily 11×34 implied volatility surface σ_{imp} into the FNO branch, the network successfully extracts latent dynamics without suffering the severe performance degradation typically observed in standard models during extreme market stress. We group the predicted results by their corresponding $\text{mean}(\sigma_{imp})$, and evaluate their respective mean absolute error (MAE). Figure 9 indicates that MEA remains tightly bounded around 0.0062 - 0.0067 across all volatility quartiles.

3.3.4. Computational efficiency benchmark

While neural networks require significant upfront training time, their inference speed is typically orders of magnitude faster than traditional numerical PDE solvers or Monte Carlo simulations. We benchmark the model's

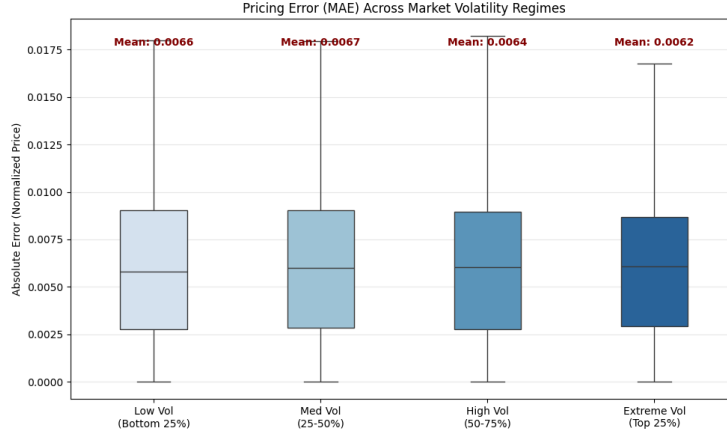


Figure 9: Mean absolute error for parts of the test set with different levels of average σ_{imp} .

computational efficiency during inference by passing the entire test dataset through the network in batched forward passes, and the total execution time is recorded. The resulting throughput, measured in 24311 options priced per second, serves to justify the deployment of FNO-DeepONet in high-frequency trading or large-scale risk management scenarios where speed is paramount.

4. Conclusions and Future Work

We developed neural operator networks, notably FNO and DeepONet, to price European call options, benchmarking them against CNO and MLP architectures. The model predicts the normalized price $\frac{V}{K}$ by utilizing moneyness $M = \frac{S}{K}$ and time to execution $\tau = T - t$ as trunk inputs, and the implied volatility surface σ_{imp} as the branch input. After fine-tuning hyperparameters and physics-informed loss weights (optimal configuration: $\lambda_{\text{data}} = 1.0$, $\lambda_{\text{PDE}} = \lambda_{\text{arb}} = 0.5$, with all boundary condition weights set to 0), the final FNO model achieved a test error of 0.000373 and an R^2 score of 0.999631. While the model struggled slightly with medium-term out-of-the-money and short-term at-the-money options, its prediction accuracy remained highly robust across different implied volatility regimes.

Future work will address two primary limitations. First, to mitigate the large relative errors caused by the dense clustering of inputs and outputs around 1 and 0 respectively, subsequent models should predict $\log(\frac{V}{K})$ using $\log(\frac{S}{K})$ as one of the inputs to be able to differentiate these clustered data. Second, the current architecture entangles the functions of Network 1 and Network 2 due to shared inputs (M and τ), preventing the extraction of true volatility estimates and meaningful Greeks. Introducing an additional loss term to force Network 2 to mimic the analytical Black-Scholes solution for a fixed σ will isolate Network 1 as a pure, dedicated volatility estimator, thereby enabling the accurate extraction of the Greeks from the trained model.

Source Code

The source code in this work can be found on Github:

<https://github.com/zefangli/Physics-Informed-Neural-Operators-for-Option-Pricing>

References

- [1] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nature Machine Intelligence* 3 (3) (2021) 218–229. doi:10.1038/s42256-021-00302-5.
- [2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, in: *International Conference on Learning Representations (ICLR)*, 2021, pp. 1–18.
- [3] B. Raonić, R. Molinaro, T. De Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, E. de Bézenac, Convolutional neural operators for robust and accurate learning of pdes, *arXiv preprint arXiv:2302.01178* (2023).
- [4] N. I. Alonso, A. Bueno-Guerrero, et al., Neural operators in finance: Mathematical framework and applications, Alberto, *Neural Operators in Finance: Mathematical Framework and Applications* (September 02, 2025) (2025).
- [5] S. Lee, J. Huh, S. Jeong, Deeponet-based surrogate modeling for bond option pricing, *AIMS Mathematics* 11 (3) (2026) 5853.
- [6] Z. Elbayed, A. Qadi el Idrissi, A hybrid pinns-fno approach for advanced option pricing models, in: *International Conference on Digital Technologies and Applications*, Springer, 2025, pp. 561–572.